

If we instead write 'x = 0', it will generate a compiler error because x is not a variable of type int, but a pointer of type *int.

'&' is used to find the address of a variable. '&x' returns a pointer to an int because x is an int. To modify the original variable, we should use '&x'.

Here is an example that describes the best use of the '*' and '&' operators:

```
package main

import "fmt"

func main() {
    var test int= 10
    /* actual variable declaration */
    var address *int
    /* pointer variable declaration */

    address = &test
    /* store address of a in pointer variable*/

    fmt.Printf("Address of test variable: %x\n", &test )

    /* address stored in pointer variable */
    fmt.Printf("Address stored in address variable: %x\n",
address )

    /* access the value using the pointer */
    fmt.Printf("Value of *address variable: %d\n", *address )
}

//OUTPUT
Address of test variable: 10428004
Address stored in address variable: 10428004
Value of *address variable: 10
```

The Go command line argument

Command line arguments are a way to execute programs by passing the parameters along to the OS. Args provides access to command line arguments. Note that the first value is the path to the program, and then it will pass all the parameters to the program. `os.Args[1:]` holds the parameters that are being passed to the program. We can get individual parameter values by using normal indexing.

```
package main

import "os"
import "fmt"

func main() {

    paramsWithProg := os.Args
```

```
WithoutProg := os.Args[1:]

param := os.Args[2]

fmt.Println(paramsWithProg)
fmt.Println(paramsWithoutProg)
fmt.Println(param))

//OUTPUT
$ go build test-cmd-args.go
$ ./ test-cmd-args w x y z
[./test-cmd-args w x y z]
[w x y z]
x
```

Go commands

The syntax is `go command [arguments]`.

Command	Usage
build	Compiles packages and dependencies
clean	Removes object files
doc	Shows documentation for packages or symbols
env	Prints Go environment information
fix	Runs Go tool fixes on packages
fmt	Runs <i>gofmt</i> on package sources
generate	Generates Go files by processing the source
get	Downloads and installs packages and dependencies
install	Compiles and installs packages and dependencies
list	Lists packages
run	Compiles and runs Go programs
test	Tests packages
tool	Runs the specified Go tool
version	Prints the Go version
vet	Runs the Go tool 'vet' on packages

You can use the 'go help [command]' for more information on any command. 

References

- [1] <http://www.golang-book.com/books/intro/>
- [2] https://golang.org/doc/effective_go.html#introduction
- [3] <http://www.tutorialspoint.com/go/>
- [4] <http://blog.smartbear.com/programming/an-introduction-to-the-go-language-boldly-going-where-no-man-has-ever-gone-before/>
- [5] <https://golang.org/cmd/go/>
- [6] <https://sendgrid.com/blog/intro-to-go-programming-language/>

By: Palak Shah

The author is a senior software engineer. She loves to explore new technologies and learn innovative concepts, and is also fond of philosophy. She can be reached at palak317@gmail.com.